ESSENTIALS OF ARTIFICIAL INTELLIGENCE

INDIVIDUAL CONTRIBUTION SHEET

Student Name	M Uvaram	**Registration No.**	D/BSE/25/0012
Degree / Intake	BSE / 42	**Group No.**	40
Project Title	**AI-Powered Dynamic Public Transport Fleet Optimizer (Route 100)**		

Technical Contributions:

Task / Component	My Contribution	Tools / AI Technique Used	Evidence / Output
MLP Model Architecture & Design	Designed and implemented the 4-layer PyTorch neural network DemandPredictorMLP (Input 23 → 128 → 64 → 32 → 1). Integrated BatchNorm1d after dense layers to stabilize internal covariate shift across heterogeneous transit features, and Dropout (0.2) with ReLU activations to prevent co-adaptation and mitigate overfitting on dense passenger counts.	PyTorch (torch.nn), Multilayer Perceptron (MLP), Batch Normalization, Dropout Regularization	train_model.py (DemandPredictorMLP class); backend/models/demand_predictor.pth
Feature Pipeline & Leakage Prevention	Built a leak-proof tabular preprocessing pipeline via ColumnTransformer. Scaled 11 continuous/calendar features using MinMaxScaler and encoded 3 nominal features (direction, stop_name, weather) with OneHotEncoder. Enforced strict chronological 80/20 train-test splitting by date so the model never trains on future passenger counts.	scikit-learn (ColumnTransformer, MinMaxScaler, OneHotEncoder), pandas, joblib	train_model.py (time_based_split); exported artifact models/preprocessor.pkl
Model Optimization & Robust Training	Configured and executed model training using Huber loss (Smooth L1) for quadratic convergence on typical loads while penalizing extreme rush-hour passenger surges linearly (preventing gradient explosion). Utilized Adam optimizer (lr = 0.001, batch size 512) and an EarlyStopping routine (patience 10) to checkpoint the best model weights.	PyTorch (optim.Adam, nn.HuberLoss), Early Stopping, Checkpoint Management	models/demand_predictor.pth (state_dict + input_dim: 23); training log
Multi-Slice Model Evaluation	Conducted rigorous holdout evaluation across N = 158,622 test records, measuring MAE, RMSE, and R^2. Computed granular slice evaluations across all 8 Galle Road stops, AM/PM peak hours, weather conditions (Clear vs Rain), and calendar types (term days vs holidays) to verify corridor-wide robustness.	scikit-learn (metrics), pandas, NumPy, JSON serialization	models/metrics.json (Overall: MAE 225.8, RMSE 519.3, R^2 0.331; stop & peak error matrices)

Model Serving & System Verification	Integrated the trained PyTorch model and preprocessor into the FastAPI backend service. Engineered an asynchronous @asynccontextmanager startup loader and vectorized batch inference pipeline (torch.no_grad()) delivering sub-millisecond predictions. Validated system reliability through an automated 7-scenario integration test suite.	FastAPI, PyTorch (Vectorized Inference), joblib, TestClient	main.py (_load_artifacts, process_and_predict); test report in test_verification.py (7/7 passed)

Declaration

I confirm that the contributions listed above accurately represent my individual work in this group project, and I can explain or demonstrate the stated work if required.

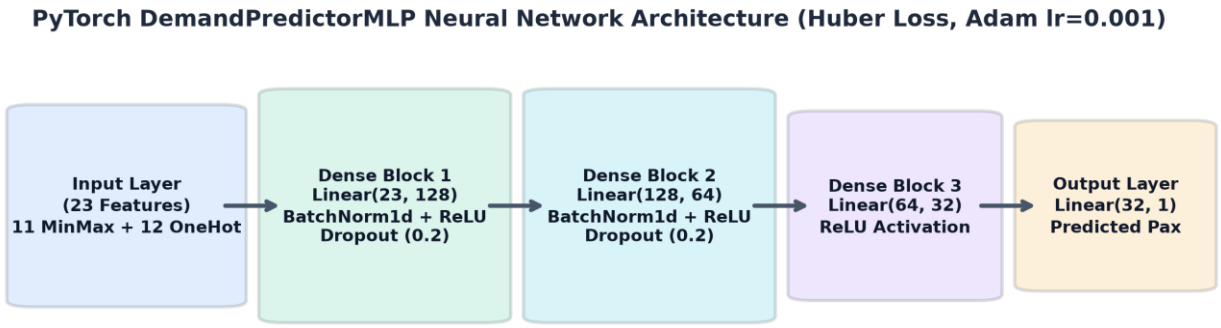
Student Signature: _____ **Date:** 18/10/2026

Evidence may include source code, model/notebook, diagram, dataset work, test results, screenshots, report section or presentation slides.

APPENDIX: TECHNICAL EVIDENCE & SYSTEM ARTIFACTS

Student: M Uvaram (D/BSE/25/0012) · Group 40 · Route 100 Transit Fleet Optimizer

Evidence 1: PyTorch MLP Architecture Implementation (train_model.py)




```
class DemandPredictorMLP(nn.Module):
    def __init__(self, input_dim: int = 23):
        super().__init__()
        self.network = nn.Sequential(
            nn.Linear(input_dim, 128),
            nn.BatchNorm1d(128),  # Mitigates internal covariate shift across features
            nn.ReLU(),
            nn.Dropout(0.2),       # Regularization against passenger count overfitting
            nn.Linear(128, 64),
            nn.BatchNorm1d(64),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(64, 32),
            nn.ReLU(),
            nn.Linear(32, 1)       # Predicts continuous passenger count per stop
        )

    def forward(self, x):
        return self.network(x)
```

Evidence 2: Chronological 80/20 Train-Test Split & ColumnTransformer (train_model.py)

```
def time_based_split(df: pd.DataFrame, test_size: float = 0.2):
    """Strict chronological split by date: past dates -> train, future -> val."""
    unique_dates = sorted(df["date"].unique())
    cutoff_idx = int(len(unique_dates) * (1 - test_size))
    train_dates = set(unique_dates[:cutoff_idx])
    return df[df["date"].isin(train_dates)], df[~df["date"].isin(train_dates)]

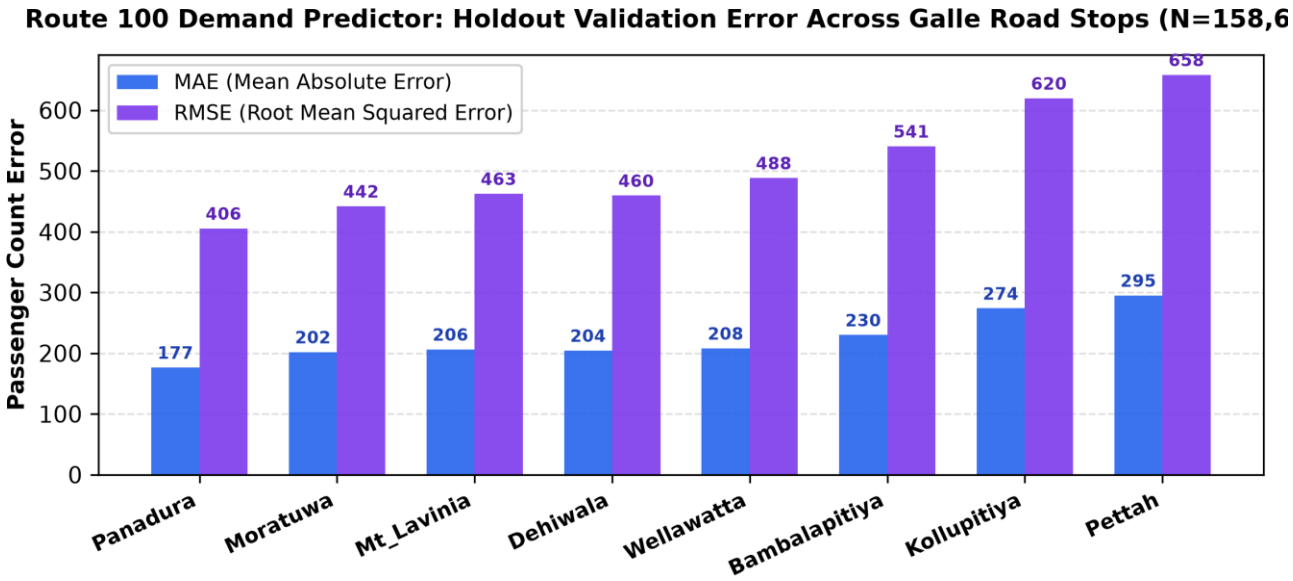
# Preprocessing: Fit ONLY on training partition to eliminate temporal data leakage
preprocessor = ColumnTransformer(transformers=[
    ("num", MinMaxScaler(), ["hour", "day_of_week", "is_weekend", "is_public_holiday",
                             "is_school_rush", "is_am_peak", "is_pm_peak", "month", ...]),
    ("cat", OneHotEncoder(handle_unknown="ignore"), ["direction", "stop_name", "weather"])
])
X_train_proc = preprocessor.fit_transform(X_train)  # Generates 23-dim feature tensor
```

Evidence 3: Model Training Hyperparameters & Saved Checkpoints

Training Dataset Size	641,378 rows (80% chronological partition)
Validation Dataset Size	158,622 rows (20% chronological holdout partition)

Loss Function	Huber Loss / Smooth L1 (robust against peak rush surge outliers)
Optimization Algorithm	Adam Optimizer (learning rate = 0.001, batch size = 512)
Early Stopping & Best Epoch	Stopped at Epoch 31 (Patience 10); Best Val Huber Loss: 225.27
Input Dimensions	23 features (11 MinMax scaled numerical + 12 One-Hot encoded categorical)
Exported Model Artifact	backend/models/demand_predictor.pth (state_dict + input_dim: 23)

Evidence 4: Holdout Evaluation Error Across Stops & Slices (models/metrics.json)



Slice / Condition	Sample Count (N)	MAE (pax)	RMSE (pax)	R^2 Score
Overall Holdout Partition	158,622	225.8	519.3	0.331
Weather: Clear vs Rain	132,626 / 25,996	196.6 / 374.4	474.7 / 704.0	0.260 / 0.324
Weekday vs Weekend	124,554 / 34,068	265.3 / 81.3	580.6 / 152.0	0.308 / 0.316
AM Peak vs School Rush	17,992 / 16,817	447.0 / 577.9	818.0 / 929.2	0.110 / 0.046

Evidence 5: End-to-End System Integration & 7-Scenario Test Run (test_verification.py)

```
=== 1. Health / Root Endpoint ===
Service: Route 100 Transit Optimizer | Framework: PyTorch MLP | Input dim: 23 | Stops: 8 stops

=== 2. Single Predict Endpoint ===
Predicted single stop (Kollupitiya 08:00): 619.6 pax (Verified: pax > 0)

=== 3. Batch Predict Endpoint ===
Batch predictions count: 2 (Panadura: 339.9, Pettah: 493.1 pax)

=== 4. Dispatch Scenario A: Weekday 08:00 AM Peak (Rush Hour) ===
Calendar Day: Wednesday (Weekend: 0, Holiday: 0) | Peak Period: am_peak
Peak Load: 619.58 at stop: Kollupitiya | Buses Needed: 8 | Headway: 8 minutes (Rush headway <= 15 min)
Corridor ETA: 112.5 min (Peak congestion multiplier: 1.25x)

=== 5. Dispatch Scenario B: Off-Peak / Night (02:00) ===
Night Peak Load: 7.37 pax | Buses Needed: 1 | Headway: 30 minutes (Clamped at max 30 min)

=== 6. Dispatch Scenario C: Poya Day Holiday Override ===
Poya Holiday Type: poya | Peak Load: 170.92 pax | Headway: 20 minutes (Lower demand on holiday)
```

```
=== 7. Dispatch Scenario D: Special Event Override ===
Event Peak Load: 1082.21 pax (vs Normal: 619.58 pax) | Buses Needed: 14 (Extreme demand surge
verified)

>>> ALL 7 SYSTEM VERIFICATION CHECKS PASSED PERFECTLY! <<<
```